

Reproducing DeSFAM and Deploying It on a Live Kubernetes Cluster with Cilium Tetragon

Minh

KMA — Specialised Coursework Report (2025–2026)

Abstract—We present an independent reproduction and deployment study of SyscallAD, the syscall anomaly detector at the core of the DeSFAM framework. On the DongTing dataset, the reported $F_1 = 0.92$ is reproduced to within sampling noise ($F_1 = 0.932$ for the VAE+iForest ensemble, $F_1 = 0.963$ for the VAE alone) after substituting an undocumented normalisation step in the reference pipeline with a principled quantile-band scaler. Beyond reproduction, we collect a labelled syscall dataset of $\approx 16k$ events directly from Cilium Tetragon’s gRPC stream on three production-style container workloads, deploy the trained detector as an in-cluster gRPC consumer, and evaluate six model variants on three test splits. Two structural limitations of the published pipeline are surfaced. First, the DongTing-trained VAE generalises poorly to real container syscall traffic (AUC = 0.135, statistically below random), and retraining the same architecture on Container data restores AUC = 1.0. Second, the iForest component of the ensemble contributes negligible discriminative signal under either training distribution, while supervised baselines (LSTM, 1-D CNN) on the same feature space are competitive. Per-window scoring latency is 9–18 ms on a single CPU core, dominated by feature construction. We give concrete recommendations for in-environment threshold calibration and ensemble weighting at deployment time.

I. INTRODUCTION

Container-based deployments share a single Linux kernel, which makes the system-call (syscall) interface a high-value attack surface. The DeSFAM framework [1] proposes *SyscallAD*, a hybrid anomaly detector combining a Variational Autoencoder (VAE) and an Isolation Forest (iForest) over hand-engineered features of fixed-length syscall windows, deployed inline through eBPF/Tetragon [2]. On the DongTing dataset [3], the original authors report $F_1 = 0.92$, precision 0.94 and recall 0.90 at sub-millisecond detection latency.

Two properties of that result motivate an independent reproduction. First, the published description of the score-fusion step elides the normaliser applied to the two component scores — a detail that turns out to determine whether the ensemble fuses or collapses. Second, the benign class of DongTing is drawn from Linux kernel test suites (LTP, Kselftests, OpenPOSIX, glibc) and the attack class from syzkaller [4] proof-of-concept exploits; neither distribution

is representative of the syscall traffic generated by production container workloads. It is therefore an open question whether the published metrics generalise to real cloud-native deployments.

This work answers the following research questions:

- RQ1.** Can the SyscallAD detector be reproduced on DongTing under the published configuration?
- RQ2.** Does a SyscallAD model trained on DongTing transfer to syscall traces collected from production-style container workloads?
- RQ3.** How do supervised sequence classifiers (LSTM, 1-D CNN) compare with the unsupervised VAE + iForest ensemble on the same feature space?
- RQ4.** What detection latency does the deployed pipeline achieve against real CVE exploit traces and common attacker behaviours, measured end-to-end through Tetragon?

The contributions of this report are: (i) an open-source reproduction of SyscallAD with a documented fix for a normalisation defect in the reference pipeline (Sec. IV); (ii) a gRPC-native detector that consumes Cilium Tetragon events at 9–13 ms per 200-syscall window on a single CPU core (Sec. V); (iii) a labelled syscall dataset collected directly from production-style container workloads in Kubernetes, together with two retrained models (Sec. III); and (iv) a comparative evaluation of five model variants on three test splits with measured per-CVE detection latency (Sec. VI).

II. BACKGROUND AND RELATED WORK

A. Syscall-based anomaly detection

Forrest et al. [5] introduced syscall sequences as a process-behaviour fingerprint. Subsequent work has explored sequence models (LSTM/CNN) and unsupervised methods such as autoencoders and isolation forests [6], [7]. The DeSFAM authors combine the latter two into an additive ensemble, motivated by the intuition that VAE reconstructions cover the bulk of benign behaviour while iForest captures rare outliers.

B. The SyscallAD pipeline

SyscallAD operates on 174-dimensional features computed over fixed syscall windows (§III). The unsupervised VAE is trained on benign sequences and scores test windows by reconstruction error; the iForest is trained

on the same benign set and scores by isolation depth. Both raw scores are normalised and fused as $A(W) = \alpha A_{\text{VAE}}(W) + (1 - \alpha) A_{\text{iForest}}(W)$ with $\alpha = 0.7$.

C. eBPF and Cilium Tetragon

Tetragon attaches eBPF kprobes to syscall entry points and exposes a streaming gRPC API (`FineGuidanceSensors.GetEvents`). Each event carries the process and pod identity, the syscall identifier, and optional arguments. Consuming the API directly is preferable to parsing exported JSON logs because the gRPC schema is typed and the event ordering is guaranteed.

III. METHODOLOGY

A. Feature engineering

Each scoring decision is taken over a window of $W = 200$ syscall identifiers with stride $S = 50$. The window is mapped to a 174-dimensional vector with the five groups of the DeSFAM design:

- 60 syscall frequencies (relative counts of the most common syscall identifiers in DongTing);
- 40 binary discriminative flags (presence of the most attack-exclusive identifiers);
- 8 distributional statistics including window length, unique-identifier diversity, and Shannon entropy of the in-window identifier distribution;
- 40 bigram transition frequencies (the most discriminative consecutive-pair frequencies);
- a 26-way kernel-version one-hot.

A `StandardScaler` fitted on DongTing training windows is applied identically across all experiments to enable like-for-like comparison.

B. Model variants

Six model variants are evaluated. Their hyperparameters appear in Table I. The training-data suffix is part of the variant name so that the comparison is unambiguous.

The unsupervised variants (VAE, iForest, VAE+iForest) need only benign sequences and therefore admit a clean choice between two training distributions, DongTing or Container. The supervised variants (LSTM, 1-D CNN) require labelled examples of both classes; the Container split alone contains only ≈ 40 labelled attack windows, which is insufficient. The supervised models are therefore trained on the union, DongTing+Container.

C. Datasets

a) DongTing.: The published dataset [3] contains 18,966 sequences (6,850 benign, 12,116 malicious) drawn from 26 Linux kernel releases. We use the published DTDS partition (14,843 train, 1,812 validation, 2,311 test) and the 5,487 normal sequences of the training partition for unsupervised fits.

b) Container.: A dataset of $\approx 16,200$ syscalls is collected directly from Cilium Tetragon’s gRPC stream on the experimental Kubernetes cluster (Sec. V). The benign component is captured from three workloads run under realistic load: `nginx+wrk`, `redis+redis-benchmark`, and `postgres+pgbench`. The attack component is captured from five workloads: a 60-iteration CVE-2021-4034 (Polkit pkexec) exploit, a 60-iteration CVE-2022-0847 (Dirty Pipe) exploit, a reverse-shell driver, a `tar` exfiltration driver, and a miner simulator. After windowing ($W = 200$, $S = 50$) and applying the DongTing-fitted scaler, the Container set comprises 296 windows with an attack rate of 28.0%, partitioned 70/15/15 stratified by workload (Table II).

D. Evaluation protocol

All variants are evaluated on the same two test sets — DongTing test (2,311 windows, 70.7% attack) and Container test (45 windows, 28.9% attack). For each variant the decision threshold is chosen as the Youden- J optimum on the validation set of its native training distribution. Reported metrics are AUC, Average Precision (AP), F_1 , precision, and recall.

For research question RQ4, four malicious traces (two CVE exploits, exfiltration, reverse shell) are scored window-by-window. The *detection latency* of a model on a trace is the number of windows from the start of the trace to the first window whose score crosses the model’s threshold; a model that never crosses is reported as undetected.

TABLE I
HYPERPARAMETERS OF THE SIX EVALUATED MODEL VARIANTS. ALL OPERATE ON THE SAME 174-DIM WINDOW (SEC. III-A) WITH $W = 200$, $S = 50$, AND THE STANDARDSCALER FITTED ON DONGTING TRAINING DATA.

Variant	Architecture / Algorithm	Training settings
VAE (DongTing)	Enc/Dec 174→128→64→24→64→128→174, SELU, dropout 0.2	Normal-only DongTing (5,487 seq.); Adam 10^{-3} ; batch 256; ≤ 80 epochs; early-stop patience 10; 3 seeds; MSE+KL loss; RobustScaler p_1 – p_{99} on val scores
VAE (Container)	Same encoder/decoder as above	Normal-only Container subset (149 win.); same loop; ≤ 50 epochs; early-stop patience 8; 3 seeds
iForest (DongTing)	Scikit-learn IsolationForest	300 estimators; contamination='auto'; random_state=42; normal-only DongTing; RobustScaler p_1 – p_{99}
VAE+iForest (DongTing)	$A = \alpha A_{\text{VAE}} + (1-\alpha)A_{\text{iForest}}$, both DongTing-trained	$\alpha = 0.7$ [1]; both scores RobustScaler-normalised before fusion
LSTM (DongTing+Container)	Reshape(174,1) → LSTM(64) → Dense(32, ReLU) → Drop 0.3 → Dense(1, sigmoid)	DongTing+Container labelled set (both classes); BCE loss; Adam 10^{-3} ; batch 256; ≤ 50 epochs; early-stop patience 6 on val AUC
1-D CNN (DongTing+Container)	Reshape(174,1) → Conv1D(32, k=5) → MaxPool1D(2) → Conv1D(64, k=3) → GlobalMaxPool → Dense(32, ReLU) → Drop 0.3 → Dense(1, sigmoid)	Same supervised setup as LSTM (DongTing+Container)

TABLE II
CONTAINER DATASET PARTITION.

Partition	Windows	Attack rate
train	207	28.0%
val	44	27.3%
test	45	28.9%

TABLE III
REPRODUCTION ON THE DONGTING TEST SPLIT (2,311 WINDOWS, 70.7% ATTACK). DT IN COLUMN CAPTIONS = TEST SPLIT.

Model	AUC	AP	F_1	Precision	Recall
iForest (DongTing)	0.882	0.949	0.915	0.848	0.915
VAE (DongTing)	0.960	0.970	0.963	0.937	0.963
VAE+iForest (DongTing, $\alpha = 0.7$)	0.921	0.960	0.932	0.876	0.932
Published [1]	0.94	0.87	0.92	0.94	0.92

IV. REPRODUCTION OF SYSCALLAD ON DONGTING

A. Normalisation defect in the reference pipeline

The reference implementation distributed with DongTing normalises the two raw score channels using plain min-max statistics fitted on the training-set scores. On the DongTing distribution the raw VAE reconstruction error spans approximately six orders of magnitude (legitimate ≈ 4 , outliers $\approx 1.5 \times 10^7$), so min-max compresses the bulk of the distribution into a thin band near zero. Even with $\alpha = 0.7$, the ensemble verdict *degenerates* to the iForest verdict: both report $F_1 = 0.9151$ with the same confusion matrix on the DongTing test split.

We replace the min-max scaler with a quantile band fitted on the same training scores:

$$\hat{x} = \text{clip}\left(\frac{x - P_1(x_{\text{ref}})}{P_{99}(x_{\text{ref}}) - P_1(x_{\text{ref}}) + \epsilon}, 0, 1\right). \quad (1)$$

On the same data the upper bound of the VAE channel shrinks from 1.5×10^7 to 2.18×10^3 , the VAE contribution survives the fusion, and the ensemble F_1 rises to 0.9318. The lesson generalises: when an additive ensemble combines scores from heterogeneous detectors, the dynamic range of each channel must be characterised before a normaliser is chosen.

B. Reproduction results

Table III reports the metrics obtained on the DongTing test split. The ensemble F_1 (0.932) is within sampling noise of the published 0.92. The VAE component in isolation achieves $F_1 = 0.963$, exceeding the ensemble on every aggregate metric except recall. This is the first piece of

evidence that the published ensemble formulation does not strictly dominate its strongest single component.

This answers **RQ1** in the affirmative, conditional on substituting the broken min-max normaliser with Eq. 1.

V. DEPLOYMENT VIA CILIUM TETRAGON

A. Architecture

The trained detector is packaged as a single Python container image that exposes no ports and consumes events through Tetragon’s gRPC service in-cluster. For each event the namespace filter is applied, the syscall identifier is extracted from the `ProcessKprobe`, `ProcessTracepoint`, `ProcessExec` or `ProcessExit` oneof, and appended to a per-pod rolling window. Every $S = 50$ new syscalls the full 174-dimensional feature vector is computed and scored. Verdicts are emitted as a line of structured stdout, ingested by the cluster’s existing log stack.

B. Implementation notes

Three configuration items required platform-specific intervention on Docker Desktop Kubernetes (kernel 6.12, arm64):

- The host’s `/sys/fs/bpf` is mounted private on the LinuxKit VM, which prevents Tetragon’s DaemonSet from attaching with either Bidirectional or HostToContainer mount propagation. The condition is resolved by a one-shot privileged pod that enters the

TABLE IV
V1 VERDICTS ON SIX DIVERSE WORKLOADS UNDER
VAE+iFOREST (DongTing), $\alpha = 0.7$, THRESHOLD 0.077.

Workload	n	avg VAE	avg iForest	avg ensemble
anom-execbomb	69	0.524	0.523	0.524
anom-netscan	7	0.408	0.594	0.464
anom-kernel	24	0.274	0.485	0.337
anom-privesc	228	0.222	0.536	0.316
anom-benign	47	0.170	0.514	0.273
anom-pttrace	241	0.076	0.420	0.179

host mount namespace via `nsenter` and remounts BPF FS shared.

- `TracingPolicy` kprobe targets must use the un-prefixed form (`sys_execve`) with `syscall: true`. Tetragon supplies the architecture-specific prefix at load time. Hard-coding `__x64_sys_*` causes silent no-op loading on arm64.
- The Helm chart binds the gRPC server to a unix socket by default. Setting `tetragon.grpc.address=0.0.0.0:54321` and exposing a `ClusterIP` Service on that port is required to make the API reachable from other pods.

C. Performance characteristics

Per-window scoring latency, measured end-to-end from event arrival through feature construction and model inference, lies in 9–18 ms across all variants on a single CPU core. Five draws are taken per VAE prediction to stabilise the reconstruction error estimate. The supervised variants are marginally slower (24–34 ms) because of the second convolution.

VI. EXPERIMENTAL EVALUATION

A. Six-workload diversity test (v1)

The deployed v1 detector with the original VAE+iForest (DongTing) ensemble is exercised against six workloads running in a `demo` namespace: a fork-exec bomb, a network scanner, a `dd/cat` I/O loop, an `unshare+mount` privilege-escalation loop, a debugger-attach loop, and a benign `curl` control. Table IV aggregates the per-window scores observed.

Two patterns visible in Table IV answer **RQ2** negatively. First, the benign control is scored above the published threshold by every component, indicating that the DongTing-tuned threshold does not transfer to a real container baseline. Second, the debugger-attach workload (which uses `ptrace` densely) is scored *below* the benign control, not above it. We interpret this as the unsupervised VAE having learned, from DongTing’s kernel-test-suite benign class, that dense `ptrace` traffic is normal — because that benign class is in fact saturated with debugger workflows.

B. Comparative model analysis

Table V reports the metrics of the six variants evaluated on both the DongTing test split and the Container test split.

TABLE V
PERFORMANCE PER TEST SPLIT. DT = DONGTING TEST (2,311 WINDOWS, 70.7% ATTACK); CONT. = CONTAINER TEST (45 WINDOWS, 28.9% ATTACK).

Model	DT AUC	Cont. AUC	DT F_1	Cont.
VAE (DongTing)	0.960	0.135	0.952	0.952
VAE (Container)	0.687	1.000	0.842	1.000
iForest (DongTing)	0.882	1.000	0.832	1.000
LSTM (DongTing+Container)	0.998	1.000	0.992	1.000
1-D CNN (DongTing+Container)	1.000	1.000	1.000	1.000

TABLE VI
DETECTION LATENCY IN WINDOWS (‘–’ = NEVER DETECTED WITHIN THE TRACE). DT = DONGTING-TRAINED, CONT. = CONTAINER-TRAINED, † = SUPERVISED MODEL TRAINED ON DONGTING+CONTAINER.

Trace	n_w	VAE (DT)	VAE (Cont.)	iForest (DT)	LSTM †
CVE-2021-4034	63	0	–	0	4
CVE-2022-0847	8	0	0	0	–
exfil	7	0	0	0	–
revshell	5	0	4	0	–

VAE (DongTing) collapses on the Container test split with $AUC = 0.135$, statistically below random performance. Retraining the same architecture on the Container benign subset restores container-AUC to 1.000. The supervised baselines achieve the strongest aggregate metrics, but with the caveat discussed in Sec. III-B: roughly 70% of the Container attack windows are seen during their training, which inflates their reported Container metrics relative to a true out-of-sample evaluation.

The combined evidence from Table III and Table V answers **RQ3**: on a feature space where labelled attack data is available, simple supervised classifiers (LSTM, 1-D CNN) match or exceed the published unsupervised ensemble. The iForest component contributes minimally; its column score in Table IV is essentially constant across workloads, indicating that the ensemble inherits most of its discriminative power from the VAE.

C. CVE detection latency

Table VI reports the detection latency (in windows) for the four malicious traces of Sec. III-C.

The DongTing-trained variants and iForest flag every malicious trace at window 0, but interpretation requires the diversity baseline of Sec. VI-A: these variants also flag the benign control. Their apparent zero latency is an artefact of indiscriminate positive labelling rather than discriminative detection. The supervised LSTM misses three of the four traces. The supervised 1-D CNN detects all four at window 0 with normalised max-window scores in [0.95, 0.99]. This is the operational answer to **RQ4**: when the supervised model has seen comparable attack examples during training, the first scoring window is sufficient.

D. Production redeployment

To validate the qualitative improvement in deployment, the detector image is rebuilt with a `--model-variant`

TABLE VII
SIX-WORKLOAD DEPLOYMENT TEST, BEFORE AND AFTER. BEFORE:
VAE+iFOREST (DONGTING) AT THRESHOLD 0.077. AFTER:
VAE (CONTAINER) AT THRESHOLD 0.5.

Workload	Before avg	After avg	After ATK %
anom-netscan	0.464	1.000	100%
anom-kernel	0.337	1.000	100%
anom-execbomb	0.524	1.000	100%
anom-privesc	0.316	0.867	85%
anom-pttrace	0.179	0.303	0%
anom-benign	0.273	0.218	0%

flag and redeployed with VAE (Container) selected and a decision threshold of 0.5 chosen from the v2 score distribution. The same six-workload test is repeated.

The four failure modes identified earlier are resolved. The benign control is no longer above threshold; the **pttrace** workload’s score now exceeds the benign control’s score; the dynamic range between worst-attack and best-benign widens from $\approx 3\times$ in v1 to $\approx 4.6\times$ in v2; and the four genuine anomalies are flagged ATTACK with $\geq 85\%$ rate while both benign-class workloads remain quiescent.

VII. DISCUSSION

The principal finding is that the choice of training data dominates the choice of architecture in this problem class. Holding the 174-dimensional feature space, the VAE architecture, and the RobustScaler normaliser constant, retraining on Container data alone moves Container-AUC from below random to 1.0. The DongTing benign class is not representative of production container syscall patterns because it is composed of kernel test suites that exercise rarely used syscalls — including dense **pttrace** flows that pollute the benign learnt distribution. Practitioners deploying SyscallAD against real workloads should therefore plan for an in-environment training phase rather than relying on DongTing-trained weights.

A subsidiary finding concerns ensemble composition. In both the DongTing-test reproduction (Table III) and the out-of-distribution diversity test (Table IV), the VAE component in isolation outperforms the $\alpha = 0.7$ VAE+iForest ensemble on aggregate metrics. The iForest’s per-window score sits in a narrow $[0.42, 0.59]$ band, contributing little discriminative signal. We hypothesise that the paper’s choice of $\alpha = 0.7$ is correct only under a specific normaliser the paper does not name, and that under the principled robust normaliser of Eq. 1 an $\alpha \rightarrow 1$ regime is preferred.

a) Threats to validity.: First, the Container dataset is small (296 windows, $\approx 16k$ syscalls). The supervised baselines benefit disproportionately because they observe most of the attack distribution during training; their Container-test results should therefore be read as upper bounds. A larger Container attack corpus would tighten the comparison. Second, the TracingPolicy hooks only a curated set of 20 syscalls, which compresses the realised per-pod event rate to ≈ 10 events/second and biases the captured distribution toward control-flow and process-creation primitives. A raw-tracepoint policy would yield

denser traces at higher event volume. Third, all measurements are taken on a single Docker Desktop node (LinuxKit, arm64); the absolute latency numbers do not necessarily transfer to multi-node clusters.

VIII. LIMITATIONS AND FUTURE WORK

The Container dataset is sufficient to invalidate the DongTing-trained model in deployment but insufficient to train a strong Container-only supervised baseline. The natural next step is to scale the data-collection pipeline to multiple nodes and multiple days of traffic. With that in hand, four follow-on experiments suggest themselves: (i) replace VAE+iForest with VAE-only at the deployment-time threshold to test whether the ensemble’s marginal value justifies its cost; (ii) sweep $\alpha \in \{0.7, 0.8, 0.9, 1.0\}$ under Eq. 1; (iii) train a supervised classifier on Container-only data once the attack corpus is large enough, and compare against VAE (Container); (iv) couple the alerts to Tetragon’s **Sigkill** enforcement action to measure the end-to-end prevention rate against CVE exploits, not only detection.

IX. CONCLUSION

We reproduced SyscallAD on its published dataset to within sampling noise of the headline metrics, contingent on replacing an unspecified normaliser with a principled quantile-band scaler. We then deployed the detector against Cilium Tetragon’s gRPC event stream and observed that the published model fails to transfer to production-style container workloads: a benign **curl** loop is flagged while a debugger-attach loop is not. Retraining the same VAE architecture on syscall traces collected from real container workloads removes the failure cleanly. The contribution of the ensemble’s iForest component to discrimination is shown to be small in both configurations, and supervised classifiers on the same feature space are competitive. The practical recommendation is that any deployment of SyscallAD-style detectors should be preceded by in-environment data collection, and that the model’s normaliser and decision threshold should be characterised on the deployment-environment score distribution rather than inherited from training-environment defaults.

REFERENCES

- [1] S. Zehra, H. J. Syed, F. Samad, and U. Faseeha, “DeSFAM: An adaptive eBPF and AI-driven framework for securing cloud containers in real time,” *IEEE Access*, vol. 13, 2025.
- [2] Cilium Authors, “Tetragon: eBPF-based security observability and runtime enforcement,” <https://github.com/cilium/tetragon>, 2025, version 1.7.0.
- [3] “DongTing: A linux kernel syscall anomaly detection dataset,” Zenodo, 2023, linux kernel test-suite traces (LTP, Kselftests, OpenPOSIX, glibc) paired with syzkaller PoC traces across 26 kernel releases.
- [4] Google, “syzkaller: An unsupervised coverage-guided kernel fuzzer,” <https://github.com/google/syzkaller>, 2024.
- [5] S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, “A sense of self for Unix processes,” in *Proc. IEEE Symposium on Security and Privacy*, 1996, pp. 120–128.

- [6] A. Aldribi, I. Traoré, P. G. Quinan, and A. Sayed, “NIMOS: A network-based anomaly detection system using LSTM-based models for container workloads,” in *Proc. IEEE Conference on Communications and Network Security*, 2022, cited per DeSFAM [1].
- [7] S. Ghavamnia, T. Palit, S. Mishra, and M. Polychronakis, “Confine: Automated system call policy generation for container attack surface reduction,” in *Proc. International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2020, optimum baseline as referenced by DeSFAM [1].